

Report LINFO1361: Assignment 1

Group N°G090 (Moodle and Inginious)

Student1: EL MANDILI DOUAA

Student2: LACHERON EDOUARD

March 18, 2026

Answer to the questions by adding text in the boxes. You may answer in either **French or English**. Do not modify anything else in the template. The size of the boxes indicate the place you **can** use, but you **need not** use all of it (it is not an indication of any expected length of answer). **Be as concise as possible! A good short answer is better than a lot of nonsense!**

1 Understanding a CP model (2 points)

1. Answer correctly to the questions 1-4. **(0.5pt)** You can help yourself by looking at <https://www.pycsp.org/documentation/constraints/> to find the definition of the different constraints.
2. Answer correctly to the question 5. **(0.5pt)**
3. Add the needed constraint to the model and submit it on INGINious. **(1pt)**

2 Minesweeper Model (8 points)

1. Implement your CP model in the file `minesweeper.py` and submit it on INGIInious. (4 pts)
2. Describe your CP model: First describe the variables of your CP model. For each set of variables, explain what they represent, give their domains and how many variables are in this set. Then, list the constraints of your CP model. For each constraint explain what it enforces, and on which variables it is applied. (4 pts)

Variables : On modélise la grille avec une matrice de variables $x[i][j]$ de taille $n \times m$.

Chaque variable correspond à une case de la grille. Le domaine de chaque variable est $\{0, 1\}$: 1 signifie qu'il y a une mine dans la case, et 0 qu'il n'y en a pas. On a donc au total $n \times m$ variables.

Contraintes : On considère uniquement les cases qui contiennent un indice (donc $clues[i][j] \neq -1$).

- **La case indice n'est pas une mine :** Une case qui contient un nombre ne peut pas être une mine, donc on impose $x[i][j] = 0$.
- **Respect de l'indice :** Pour chaque case avec un indice, on regarde toutes ses cases voisines (jusqu'à 8). La somme des variables correspondant à ces voisins doit être égale à la valeur de l'indice. Cela permet de garantir que le nombre de mines autour de la case est correct.

3 Atom Placement Problem()

1. Formulate the atom placement problem as a Local Search problem (problem, cost function, feasible solutions, optimal solutions). (1 pt)

Une solution consiste à attribuer un type d'atome à chaque site du graphe, ce qui peut être représenté par une liste.

La fonction de coût correspond à l'énergie totale des interactions entre les sites voisins. Pour chaque arête, on ajoute une valeur donnée par la matrice d'énergie selon les types d'atomes.

Une solution est faisable si tous les sites sont remplis et que le nombre d'atomes de chaque type est respecté.

Une solution optimale est celle qui minimise l'énergie totale.

2. You are given a template on Moodle: *atomplacement.py*. Implement your own extension of the *Problem* class from *aima-python3*. Implement the *maxvalue* and *randomized maxvalue* strategies. To do so, you can get inspiration from the *randomwalk* function in *search.py*. Your program will be evaluated on 15 instances (during 1 minute) of which 5 are hidden. We expect you to solve at least 12 out the 15.

- (a) *maxvalue* chooses the best node (i.e., the node with maximal value) in the neighborhood, even if it degrades the quality of the current solution. The *maxvalue* strategy should be defined in a function called *maxvalue* with the following signature:

`maxvalue(problem, limit=100)`. (2.5 pts)

Nous avons implémenté la stratégie *maxvalue* en sélectionnant à chaque étape le voisin ayant la meilleure valeur parmi ceux générés.

- (b) *randomized maxvalue* chooses the next node randomly among the 5 best neighbors (again, even if it degrades the quality of the current solution). The *randomized maxvalue* strategy should be defined in a function called *randomized_maxvalue* with the following signature:

`randomized_maxvalue(problem, limit=100)`. (2.5 pts)

La stratégie *randomized maxvalue* choisit aléatoirement un voisin parmi les meilleurs, ce qui permet d'explorer différentes solutions.

3. Compare the 2 strategies implemented in the previous question and randomwalk defined in *search.py* on the given vertex cover instances. Run the strategies during 100 steps, and report, in a table, the computation time, the value of the best solution and the number of steps needed to reach the best result. For the randomized max value and the random walk, each instance should be tested 10 times to reduce the effects of the randomness on the result. When multiple runs of the same instance are executed, report the mean of the quantities. (3 pts)

Inst.	Max value			Random max value			Random walk		
	T(s)	Val	NS	T(s)	Val	NS	T(s)	Val	NS
i01	0.14	151	6	0.15	131.0	62.2	0.043	217.7	38.9
i02	5.36	1281	18	18.14	1247.7	44.8	0.31	1711.6	56.8
i03	6.08	1128	23	22.39	1118.7	64.0	0.36	1806.6	49.0
i04	3.45	274	13	13.25	244.6	35.0	0.23	579.4	36.4
i05	0.13	304	5	0.54	297.7	51.0	0.036	365.7	53.4
i06	0.17	269	8	0.62	270.8	48.6	0.039	342.2	68.4
i07	0.81	244	15	3.06	224.3	53.2	0.11	416.4	59.5
i08	2.67	498	15	10.37	461.5	70.3	0.21	843.6	53.2
i09	0.82	481	8	3.15	410.8	43.6	0.12	677.9	26.4
i10	2.28	2011	14	2.32	1986.4	52.7	0.2	2362.0	38.8

NS: Number of steps — T: Time — Val: Value of the best solution

4. (4 pts) Answer the following questions:

(a) In one sentence, what is the best strategy? (0.5 pt)

Selon nos observations, la meilleure stratégie dans ce contexte est le RandomMaxValue.

(b) Why do you think the best strategy beats the other ones? What conclusions can be drawn on the atom placement problem ? (1.5 pt)

La stratégie "RandomMaxValue" est la meilleure selon nous car elle permet d'explorer plusieurs trajectoires de recherche différentes selon les exécutions, et donc d'éviter plus facilement certains optima locaux. À l'inverse, maxvalue est entièrement déterministe : à partir d'un même état initial, il suit toujours le même chemin et retourne souvent la même solution. Cela montre que le problème contient plusieurs optima locaux et qu'un peu d'aléatoire améliore la recherche.

(c) What are the limitations of each strategy in terms of diversification and intensification? (1 pt)

MaxValue est très orienté intensification parce qu'il choisit toujours le meilleur voisin, donc il améliore vite mais explore peu.
RandomWalk fait l'inverse : il explore beaucoup mais sans vraiment chercher à améliorer la solution.
RandomMaxValue est un bon compromis entre les deux, parce qu'il garde un peu d'aléatoire tout en restant concentré sur les bons voisins.

(c) What is the behavior of the different techniques when they fall in a local optimum? (1 pt)

Quand MaxValue arrive dans un optimum local, il reste souvent bloqué ou tourne en rond.
RandomWalk peut en sortir grâce au hasard, mais il peut aussi partir dans de mauvaises directions.
RandomMaxValue s'en sort mieux parce qu'il peut sortir de l'optimum local tout en restant proche des bonnes solutions.